

→ Authentication (vs) Authorization

↓
Identify
the
user.

masaischool.com
→ To access this website no authentication
is required.

→ Register for a course.

↳ Authentication is must.

↓
Who is the user.
→ Proof of Identity.

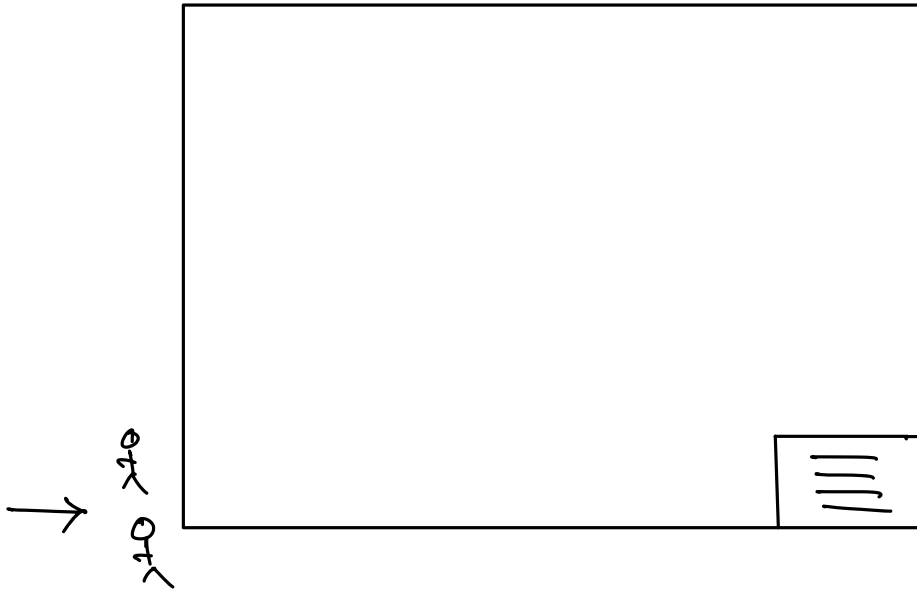
→ Email + password / otp

→ Phone + otp / password

Authorization.

↳ What the user is trying to access.

Shopping Mall.



RBAC.

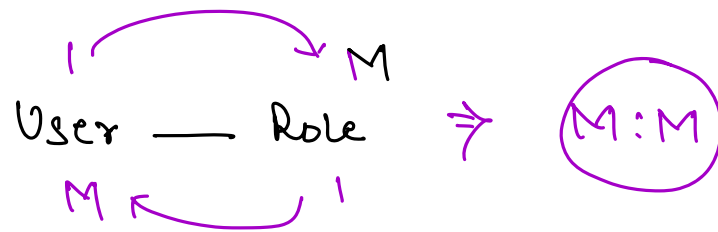
↳ Role Based Access Control.

users

id	name	email	pass

roles

id	name
1	Student
2	Instructor
3	TA
4	Admin



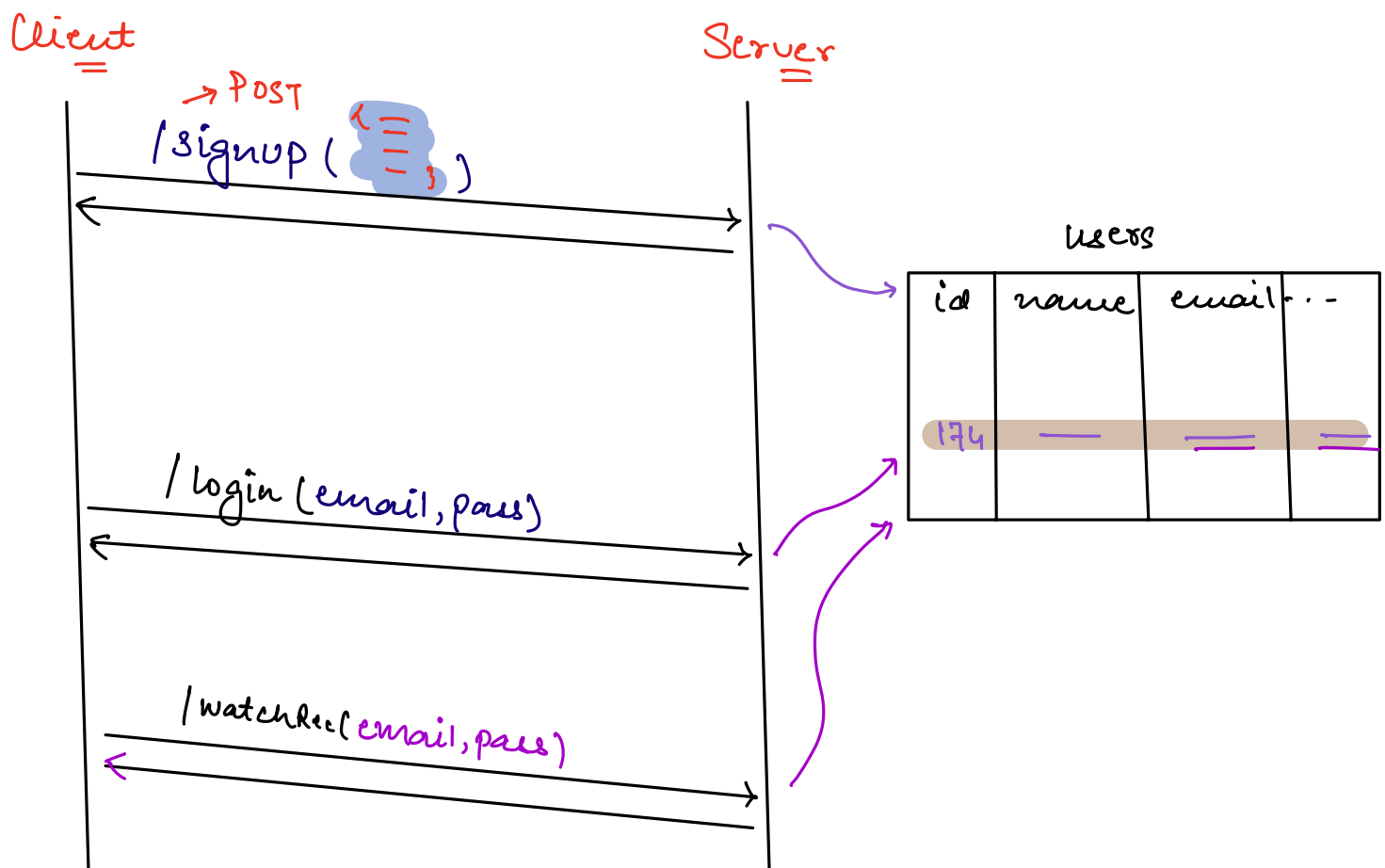
users-roles

user-id	role-id

masaischool.com/admin

→ If user is NOT authorized then
Server returns **401** http status.
↓
Unauthorized

signup & login flow



⇒ HTTP protocol is stateless.

$\Rightarrow$ Every HTTP request is independent.

⇒ Every HTTP request should be self-sufficient =

TOKEN.

Client

Server

→ POST

/signup ({
 name: '...',
 email: '...',
 pass: '...',
})

/login (email, pass)

token

token
↓
/watchRec(=)

users

id	name	email	...
174	—	—	—

tokens

id	user-id	value	expiry-date
1	174	mn9678 ...	3 rd Aug —

Amazon.in: —
massaischool.com —
fb.com: —
— : —
— : —

for every token validation,
we are making a DB call
(n/w call) which is expensive.

JWT



Self-Validating
Tokens.
=



How to store passwords in DB?

users

id	name	email	password
—	—	—	abcd
—	—	—	1234
—	—	—	admin

→ Password shouldn't be stored in the raw format in the DB.

→ Encryption

raw password $\xrightarrow[\text{Encryption Algo}]{\text{secret key}}$ encrypted password.

users

id	name	email	password
—	—	—	x!@!127948a6xy
—	—	—	
—	—	—	

→ Hashing

hash(x) ↓

==
==
=

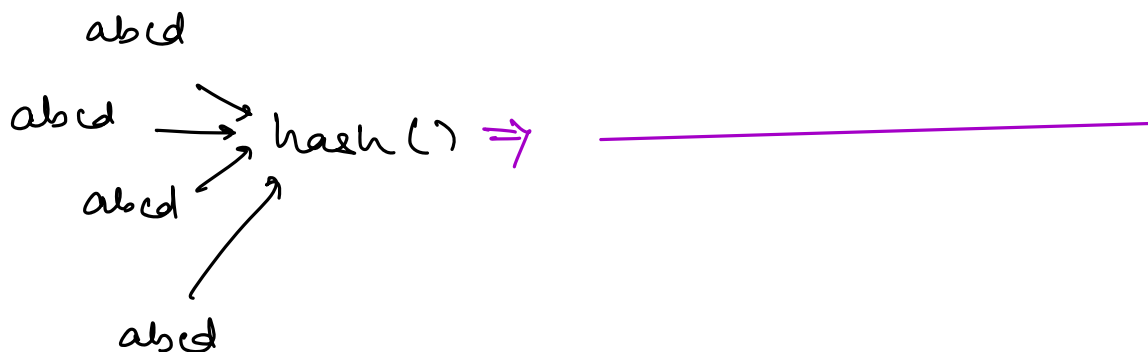
3

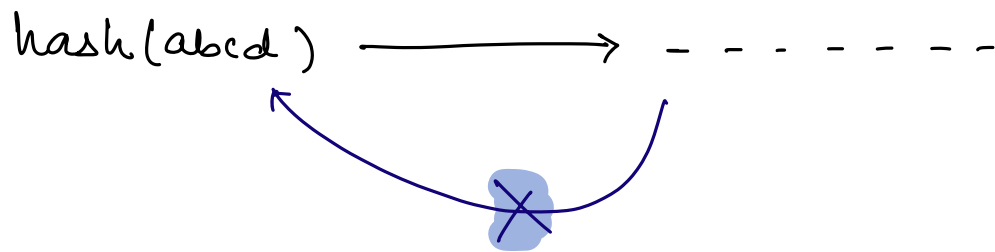
abcd
////

users

id	name	email	password
—	—	—	x!@!127948abxy
—	—	—	—
—	—	—	x!@!127948abxy
			—
			x!@!127948abxy

→ hashed
password.





hacker@ — } →
abc

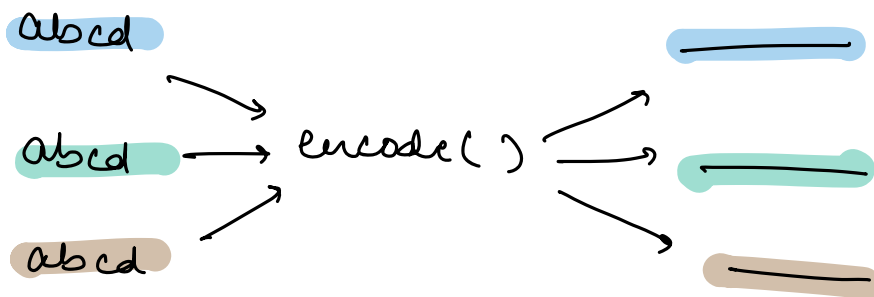
users

id	name	email	password
—	—	—	x!@!127748abxy →
—	—	—	—
—	—	—	x!@!127748abxy
—	—	—	—
—	—	hacker@ —	x!@!127748abxy

⇒ Hashing + Salting
 ↳ Randomness.

fun(x) {
 hash(x + salting)
3

BCrypt password encoder {
 encode(x) ⇒ —
 verify(—, —)



users

id	name	email	password
—	—	—	x!@!127948a6xy
—	—	—	—
—	—	—	uu?.\$#<xy7178KL—
			—

/login(email, raw-password)



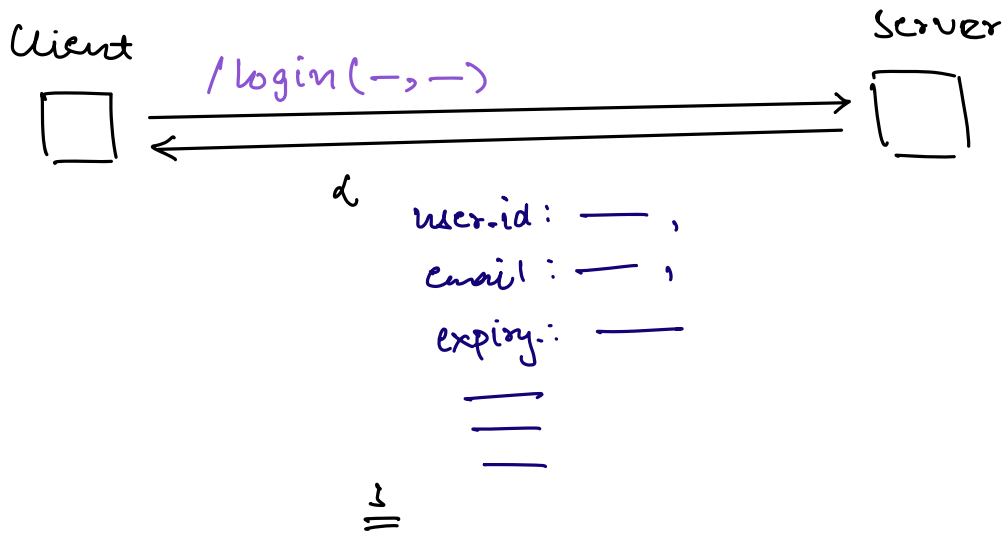
encode()



Verify(encoded-password, raw-password)



→ If it is possible to generate the
encoded password from the raw-password
or not.



JWT



JSON Web Token

d

user.id: — ,

email: — ,

expiry: —

—

—

—

}

=

decode

base64
encoding

JWT



A.B.C

Encryption
Algo.



Payload.



Signature